

AWS Identity and Access Management (IAM)

From Fundamentals to Advanced Patterns — with Activities

Naveen Kumar

Visiting Professor, IIIT Surat

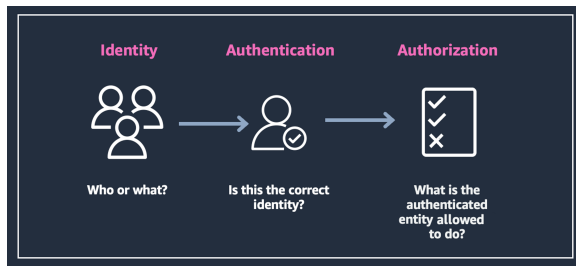
August 23, 2025

Agenda

- 1 Introduction
- 2 Core Concepts
- 3 Federation & Cross-Account
- 4 Features and Security
- 5 Best Practices and Governance
- 6 Advanced Use Cases
- 7 Case Study
- 8 Examples and Wrap-Up

What is IAM?

- AWS service for **securely managing access** to AWS resources.
- Covers **authentication** (who you are) and **authorization** (what you can do).
- Global service, **no additional cost**; you pay for the services that identities use.
- Central to least-privilege and compliance.



Why IAM is Critical

- Reduce risk via **granular permissions** and separation of duties.
- Enable **governance at scale** across accounts and teams.
- Support **auditing and compliance**: CloudTrail, Access Analyzer, Credential Reports.
- Foundation for **zero trust**: verify identity & context each request.

Analogy

Think of IAM as a **campus access system**: ID check at the gate (AuthN), and lab/library passes (AuthZ).

Authentication vs. Authorization: A Simple Analogy

Authentication (AuthN)

- **Verifies who you are.** (e.g., your driver's license)
- Confirms the identity of a user, application, or service.
- Mechanisms include: passwords, access keys, Multi-Factor Authentication (MFA), and federated Single Sign-On (SSO).

Authorization (AuthZ)

- **Determines what you can do.** (e.g., the permit on your license)
- Dictates which actions are allowed on which resources.
- Mechanisms include: IAM policies, resource policies, and permission boundaries.

The Shared Responsibility Model

AWS is responsible for security of the Cloud

- Securing the infrastructure (hardware, software, networking, facilities).
- Protecting managed services (e.g., EC2, S3, RDS).

You are responsible for security in the Cloud

- Managing your data, identities, and policies (like IAM).
- Securing your applications and network configurations.

The Key Takeaway for IAM

- IAM is a customer responsibility.
- Misconfiguration of IAM policies can lead to security vulnerabilities and data exposure.

Check Your Understanding

Who is responsible if you attach a policy with "Action": "*" and sensitive data is leaked?

Answer: The customer.

Common IAM Use Cases

1. Human Users

- **Admins, Developers, and Analysts:** Manage access for people who need to interact directly with the AWS console or CLI.
- **Federated Access:** Integrate with an enterprise Identity Provider (IdP) like Active Directory or Okta for Single Sign-On (SSO).

3. Cross-Account and Third-Party Access

- **Delegation:** Allow a user or service in one AWS account to assume a role and perform actions in another account.

2. Workload Identities

- **AWS Services:** Grant permissions to services like EC2 instances, Lambda functions, and ECS tasks to securely access other AWS resources (e.g., an EC2 instance reading from an S3 bucket).
- **Kubernetes Clusters:** Use IAM Roles for Service Accounts (IRSA) to give permissions to EKS pods.

IAM vs. Organizations vs. Identity Center

- **IAM (Identity and Access Management)**
 - Works inside a single AWS account.
 - Defines **users, groups, roles, and policies**.
 - Controls permissions at the account level.
- **AWS Organizations**
 - Works across multiple AWS accounts.
 - Enables centralized governance: **Service Control Policies (SCPs)**.
 - Provides **consolidated billing** and account management.
- **IAM Identity Center (formerly AWS SSO)**
 - Provides **single sign-on (SSO)** across accounts and apps.
 - Connects with external identity providers (e.g., Azure AD, Okta).
 - Centralized user access without creating separate IAM users in each account.

- **What are IAM Users?** Identities with long-term credentials (username/password or access keys) created inside a single AWS account.
- **Authentication Methods:**
 - **Password** – for AWS Console login.
 - **Access Keys** – for CLI/SDK access (programmatic).
 - **MFA (Multi-Factor Authentication)** – adds extra security layer.
- **Use Cases:** Typically for individual people (admins, developers) or legacy applications that cannot use roles.
- **Best Practice:** Minimize IAM users. Prefer **Federated Access (SSO)** or **IAM Roles** for workloads and temporary access.

IAM Groups

- **What are IAM Groups?** A logical collection of IAM users. Groups simplify access management by letting you assign permissions to the group instead of managing each user individually.
- **How Permissions Work:** Users automatically inherit all policies attached to the group. Example: If the “Developers” group has S3 read access, every user in that group can read from S3.
- **Best Practice:** Attach **managed policies** (AWS-managed or custom) to groups. → Ensures consistency and avoids errors from assigning permissions user by user.
- **Common Use Cases:** Organize users by job function:
 - Admins – full account access
 - Developers – deploy and manage resources
 - Auditors – read-only access
 - Support – limited troubleshooting permissions

IAM Roles: Overview

- **What is a Role?** An IAM identity that provides temporary credentials, issued through the AWS Security Token Service (STS). Unlike users, roles do not have long-term credentials.
- **Who Can Assume a Role?**
 - AWS services (e.g., EC2, Lambda)
 - Human users (via AWS Console, CLI, or Federation)
 - External principals (cross-account or third-party access)
- **Role Policies: Two Key Parts**
 - **Trust Policy** – defines who is allowed to assume the role. Example: EC2 service or another AWS account.
 - **Permissions Policy** – defines what actions the role can perform once assumed. Example: Read from an S3 bucket, write to DynamoDB.
- **Best Practice:** Use roles for workloads, cross-account access, and federation → avoid embedding permanent credentials.

Role Trust Policy (JSON Example)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {"Service": "ec2.amazonaws.com"},
      "Action": "sts:AssumeRole"
    }
  ]
}
```

- A **Trust Policy** defines who is allowed to assume this role.
- Written in JSON format, attached to the IAM Role itself.
- Required for cross-service or cross-account access.

Breaking Down the Trust Policy

- **Version:** Policy language version (always keep latest: 2012-10-17).
- **Effect:** Allow means permission is granted.
- **Principal:** Defines who can assume the role. Example: "Service":
"ec2.amazonaws.com" → EC2 service can assume the role.
- **Action:** Always sts:AssumeRole for trust policies.
- **Statement:** Container for rules (can have multiple statements).

IAM Role Use Cases

- **EC2 Instance Role** Attach a role to an EC2 instance so it can securely access AWS resources (e.g., read from S3, write to DynamoDB) without storing access keys.
- **Cross-Account Access** A role in Account A can be assumed by a user or service in Account B. Example: A CI/CD pipeline in the DevOps account assumes a role in the Production account to deploy resources.
- **Federated User Access (SSO)** A corporate employee logs in through SSO (e.g., via Okta or Azure AD). After authentication, they are mapped to an IAM Role that grants AWS permissions.
- **Service-Linked Roles** Some AWS services automatically create and use roles to perform actions on your behalf. Example: RDS using a role to publish logs to CloudWatch, or Lambda using a role to read from SQS.

- A *JSON document* that defines permissions. It specifies what actions are *allowed or denied* on which resources.
- Policies can be attached to identities (*identity-based policies*) like users, groups, and roles, or directly to resources (*resource-based policies*) like S3 buckets.
- The final permission is determined by an *evaluation logic* that considers all applicable policies, including explicit denies, which always take precedence.

IAM Policy Elements

- **Core Keys:** The essential components of any policy are 'Version', and 'Statement'. The statement itself contains:
 - **Effect:** The permission ('Allow' or 'Deny').
 - **Action:** The API calls being allowed or denied (e.g., 's3:GetObject').
 - **Resource:** The specific AWS resource(s) the action applies to (e.g., a specific S3 bucket).
 - **Principal:** The identity to which a resource policy grants access.
- **Condition Keys:** These are optional elements that add extra logic to a policy. They are used to specify conditions under which the policy statement takes effect, such as requiring MFA ('aws:MultiFactorAuthPresent') or checking a source IP ('aws:SourceIp').
- **Best Practice:** Follow the principle of least privilege. Use Amazon Resource Names (ARNs) to scope your policies as narrowly as possible, rather than using wildcards ('*').

Policy Evaluation Logic: Key Rules

- **Default: Implicit Deny** By default, all requests are denied unless explicitly allowed.
- **Explicit Allow** If a policy contains an "Effect": "Allow", the action is permitted.
- **Explicit Deny (Highest Priority)** Any "Effect": "Deny" will always override an allow statement. → Used for enforcing restrictions (e.g., blocking risky actions).

Policy Example: Requiring MFA

Explanation

This policy explicitly denies all actions for a user unless Multi-Factor Authentication (MFA) is enabled. → Demonstrates how an explicit Deny overrides other allows.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Action": "*",
      "Resource": "*",
      "Condition": {
        "Bool": {"aws:MultiFactorAuthPresent": "false"}
      }
    }
  ]
}
```

Inline vs. Managed Policies

Managed Policies

- Standalone policy object, stored separately from any user/role/group.
- **Reusable** – can be attached to multiple identities.
- **Versioned and centrally managed** – easier to update and maintain.
- Two types: **AWS Managed** (predefined by AWS) and **Customer Managed** (created by you).
- Best for shared, repeatable permission patterns (e.g., “S3ReadOnlyAccess”).

Inline Policies

- Policy is **embedded directly** into a single IAM identity (user, role, or group).
- **Identity-bound** – deleted automatically if the identity is deleted.
- Cannot be reused or shared with other identities.
- Best for unique, one-off permissions (e.g., a single auditor role with temporary S3 access).
- Harder to maintain at scale → should be used sparingly.

AWS-Managed vs. Customer-Managed Policies

AWS-Managed Policies

- Created, maintained, and automatically updated by AWS.
- **Quick start** – useful for beginners and common use cases.
- Broad and generic in scope (e.g., `AdministratorAccess`, `AmazonS3ReadOnlyAccess`).
- No direct control over changes – AWS may update them at any time.

Customer-Managed Policies

- Fully created and managed by you.
- **Tailored** to specific security and compliance needs.
- Enables the principle of **least privilege**.
- Granular control: you decide exactly what actions/resources are allowed.

Recommended Strategy

Begin with AWS-managed policies for convenience during initial setup or development. Then refine and migrate to customer-managed policies, using **CloudTrail logs and IAM Access Analyzer** to identify only the permissions actually required. → Achieves true least privilege.

Policy Versions

- Each managed policy can retain up to **5 versions** at a time.
- One version is marked as **default** (the active one).
- Older versions can be safely rolled back if a new version causes issues.
- Use versioning for:
 - **Safe rollbacks** during deployment failures.
 - **Staged hardening** – gradually tightening permissions.
 - **Change tracking** with Infrastructure as Code (IaC) and code review.
- Delete unused versions to free up space before creating new ones.

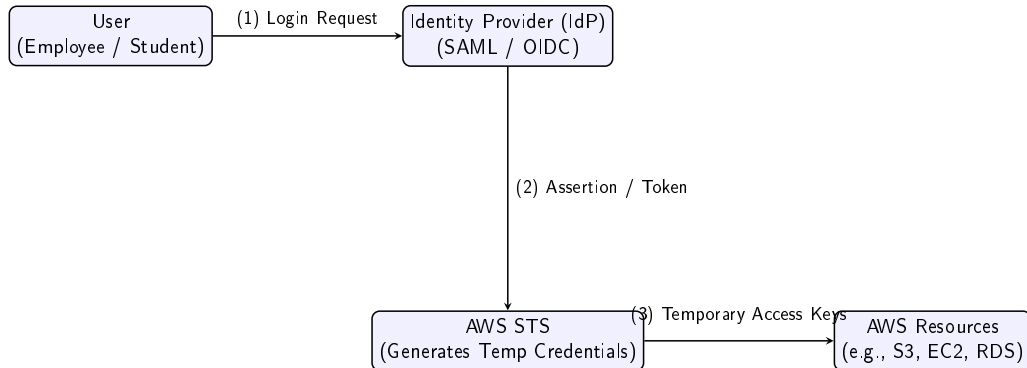
- ➊ Which two policies are attached to an IAM role?
- ➋ Which always takes precedence: Allow or Deny?
- ➌ When would you prefer Inline vs. Managed policies?
- ➍ Difference between AWS-managed and Customer-managed policies?
- ➎ How many versions can a managed policy keep?
- ➏ Purpose of a Service-linked role?
- ➐ What happens if no policy explicitly allows an action?

- ❶ **Which two policies are attached to an IAM role?** → Trust policy and Permissions policy
- ❷ **Which always takes precedence: Allow or Deny?** → Explicit Deny always overrides
- ❸ **When would you prefer Inline vs. Managed policies?** → Inline = single identity; Managed = reusable across identities
- ❹ **Difference between AWS-managed and Customer-managed policies?** → AWS-managed = broad, prebuilt; Customer-managed = fine-grained, least privilege
- ❺ **How many versions can a managed policy keep?** → Maximum 5 versions retained
- ❻ **Purpose of a Service-linked role?** → Allows AWS service (e.g., RDS, Lambda) to perform actions on your behalf
- ❼ **What happens if no policy explicitly allows an action?** → Implicit Deny — no access unless explicitly allowed

Identity Providers (IdP)

- **What is an IdP?** An Identity Provider (IdP) is a service that stores and manages user identities and passwords, and helps users log in securely to different applications.
- **SAML 2.0:** Enterprise-grade Single Sign-On (SSO) used by companies to connect with directories like AD FS, Azure AD, or Okta.
- **OIDC:** A simpler, modern protocol for logging into web and mobile apps (e.g., Amazon Cognito, Auth0, GitHub).
- **Federation Advantage:** Instead of creating separate IAM users for everyone, AWS gives temporary role-based access via STS using IdP login.
- **Why use IdPs?** They reduce password fatigue (no need to remember many logins), improve security, and centralize authentication.

Federation Flow (High-Level)



Flow: User logs in → IdP verifies → AWS STS issues temporary keys → User accesses AWS resources.

Cross-Account Access

- **Step 1:** In Account B (destination), create a role and set a **trust policy** that allows Account A to use it. (Trust policy = who can assume the role.)
- **Step 2:** Users or workloads (like EC2/Lambda) in Account A can **assume the role** in Account B and temporarily get its permissions.
- **Step 3:** This lets Account A securely access resources in Account B (e.g., an S3 bucket in B).
- For large setups, manage centrally using **AWS Organizations** and **IAM Identity Center (SSO)**.

Analogy: Like visiting another office building: your office (Account A) requests a visitor pass (role) from the other office (Account B). With that pass, you can enter only the areas allowed.

Hands-On: Assume Role via CLI

```
# Step 1: From Account A, request to assume a role in Account B
aws sts assume-role \
--role-arn arn:aws:iam::222222222222:role/CrossAccountReadOnly \
--role-session-name demo-session > /tmp/creds.json
```

Step 2: Extract the temporary credentials from the response

```
export AWS_ACCESS_KEY_ID=$(jq -r .Credentials.AccessKeyId /tmp/creds.json)
export AWS_SECRET_ACCESS_KEY=$(jq -r .Credentials.SecretAccessKey /tmp/creds.json)
export AWS_SESSION_TOKEN=$(jq -r .Credentials.SessionToken /tmp/creds.json)
```

Step 3: Use the temporary credentials to test access

```
aws s3 ls arn:aws:s3:::prod-logs-bucket --region us-east-1
```

- **Step 1:** Use 'sts assume-role' to get temporary credentials from Account B.
- **Step 2:** Save those credentials into environment variables.
- **Step 3:** Run an S3 command to confirm you can now access resources in Account B.

Granular Permissions

- IAM policies can control access at **3 levels**:
 - **Service level**: Which AWS service (e.g., S3, EC2).
 - **Resource level**: Which specific resource (e.g., a single S3 bucket).
 - **Condition level**: Extra rules (e.g., allow only if MFA is used).
- Use **ARNs (Amazon Resource Names)** to precisely identify resources instead of using wildcards '*'.
- Use **tags with ABAC (Attribute-Based Access Control)** to scale permissions.
Example: Developers with tag 'team:app1' can only access resources tagged 'team:app1'.
- **Example conditions**:
 - MFA must be enabled ('aws:MultiFactorAuthPresent').
 - Access allowed only from a certain IP range ('aws:SourceIp').
 - Restrict usage by time of day (working hours).
 - Resource must have required tags (e.g., 'environment:dev').

Key Idea: Granular permissions = Give exactly what is needed, nothing more.

Multi-Factor Authentication (MFA)

- **What is MFA?** An extra layer of security that requires something you know (password) + something you have (device/code).
- **Types of MFA devices:**
 - Virtual apps (Google Authenticator, Authy) → generate time-based codes.
 - Hardware keys (FIDO/U2F like YubiKey) → strongest option.
 - SMS codes → possible but **not recommended** (less secure).
- **How to enforce MFA:**
 - Via IAM policies (example: deny actions if MFA not present).
 - At sign-in settings (require MFA for console logins).
- **Best Practice:** Require MFA for all privileged actions such as:
 - Managing IAM users/roles.
 - Using KMS (encryption keys).
 - Billing and account management tasks.

Temporary Credentials (STS)

- **What are they?** Short-lived security tokens issued by the **AWS Security Token Service (STS)**. Example APIs: `AssumeRole`, `GetFederationToken`.
- **Why use them?** They automatically expire (default = 1 hour), so they reduce the risk of leaking long-term access keys.
- **Control options:**
 - `DurationSeconds` → how long the credentials remain valid.
 - **Session Tags** → attach metadata (like `team:dev`) for fine-grained access and auditing.
 - `ExternalId` → extra protection in cross-account access (prevents confused-deputy attacks).
- **Best Practice:** Always prefer temporary credentials for applications, CI/CD pipelines, and cross-account access instead of static IAM keys.

Resource-Based Policies

- **What are they?** Policies that are **directly attached to a resource** instead of a user or role. Example resources: S3 buckets, Lambda functions, KMS keys, SQS queues.
- **Why use them?** They allow you to specify who (which account, user, or service) can access a resource, even if the identity is outside your AWS account.
- **Common use cases:**
 - Granting another AWS account access to your S3 bucket.
 - Allowing a Lambda function to be invoked by an external service.
 - Sharing KMS encryption keys securely across accounts.
- **Best Practice:** Combine resource-based policies with **identity-based policies** for defense-in-depth. (Identity-based = permissions linked to users/roles; Resource-based = permissions linked to resources.)

Permission Boundaries

- **What is it?** A permission boundary is a special policy that sets the **maximum limit** of permissions an IAM user or role can ever get. Even if other policies grant more, the boundary blocks anything beyond the limit.
- **Why use it?** It allows safe delegation of IAM management. For example, you can let team leads create roles for their apps, but ensure they can't give away full admin rights.
- **Common scenarios:**
 - Large organizations with many teams → prevent accidental over-permissioning.
 - Multi-tenant platforms → stop one tenant from escalating into another tenant's data.
 - Training environments → give learners freedom, but keep a safety cap on privileges.

Service-Linked Roles

- **What are they?** Special IAM roles that are **automatically created and managed by AWS services** when those services need to act on your behalf.
- **Why use them?** They simplify integration because you don't need to manually create and attach roles. Each role is configured with the **least privilege** required by the service.
- **Examples:**
 - `AWSServiceRoleForEC2Spot` → lets EC2 Spot Instances request and manage spot capacity.
 - `AWSServiceRoleForRDS` → allows RDS to publish logs to CloudWatch.
 - `AWSServiceRoleForElasticLoadBalancing` → enables ELB to manage network traffic routing.

ACLs vs. IAM Policies

- **Access Control Lists (ACLs):** Legacy mechanism that attaches simple “allow lists” directly to resources. Example: An S3 object ACL that says “User X can read this file.”
- **IAM Policies:** Modern mechanism that uses JSON-based rules attached to identities (users, roles) or resources. Supports advanced features like conditions, explicit deny, and policy evaluation logic.
- **Key Differences:**
 - ACLs = old, simple, resource-only.
 - IAM Policies = flexible, powerful, and recommended for most use cases.

- **What is it?** A tool that automatically **scans IAM policies and resource settings** to detect if resources (like S3 buckets, KMS keys, or IAM roles) are shared outside your account.
- **Why is it useful?** It helps you confirm that your permissions follow the **principle of least privilege** and alerts you if something is exposed unintentionally.
- **Common findings:**
 - An S3 bucket accidentally made public.
 - A KMS key shared with another AWS account.
 - An IAM role that any external account could assume.
- **Integration:** Works with AWS policy validation tools and sends findings into workflows (e.g., AWS Security Hub, CloudWatch, or SIEM tools) for remediation.

Credential Report & CloudTrail

- **Credential Report:** A downloadable **CSV file** that gives a snapshot of IAM user security status. Includes details like: password enabled/disabled, last password change, active access keys, key rotation, and MFA enabled.
- **AWS CloudTrail:** A logging service that records every **API call** in your AWS account. Tells you who did what, when, and from where (e.g., console login from a specific IP).
- **How they work together:**
 - Credential Report → checks “who can do what” (security posture).
 - CloudTrail → shows “who actually did what” (activity logs).

Activity: Write a Safer S3 Policy

Goal: Give users **read-only** access to bucket `student-data`, but only if they are logged in with **MFA** enabled.

Policy JSON:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "S3ReadOnlyWithMFA",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:ListBucket"],
      "Resource": [
        "arn:aws:s3:::student-data",
        "arn:aws:s3:::student-data/*"
      ],
      "Condition": {
        "Bool": {"aws:MultiFactorAuthPresent": "true"}
      }
    }
  ]
}
```

Explanation:

- **Effect:** Allow → Grants permission.
- **Action** → Only `GetObject` & `ListBucket`.
- **Resource** → Just `student-data` bucket + objects.
- **Condition** → Works only if MFA is present.

Least Privilege by Design

- The fundamental security principle: grant only the permissions required to perform a specific task.
- **Iterate and refine:** Start with a broader policy for initial development, then use tools like CloudTrail to analyze access patterns and tighten permissions over time.
- **Be specific:** Avoid using wildcards ('*'). Scope your policies to specific actions and resources using ARNs and tags.
- **Layer conditions:** Use condition keys (e.g., 'aws:SourceIp', 'aws:MultiFactorAuthPresent') to further limit access based on context.

Avoid Root Account Usage

- The root account is all-powerful; its credentials should only be used for a handful of account and service management tasks.
- "Lock it away:" Immediately enable MFA on the root account and store its credentials securely. Never use the root account for day-to-day operations.
- "Create administrators:" Create a separate, highly privileged IAM user or role for administrative tasks. This provides an audit trail and limits the risk of root account compromise.

Password and Key Policies

- Enforce a "strong password policy" for IAM users, but prefer federation for human users.
- "Rotate access keys" regularly and avoid embedding them in source code.
- For secrets and credentials, use dedicated services like AWS Secrets Manager or AWS Systems Manager Parameter Store instead of hard-coding them.

MFA Everywhere

- **Why MFA?** Multi-Factor Authentication (MFA) adds a second verification factor (device or key) beyond just a password. → Protects against compromised credentials.
- **Enforce Globally:** Require MFA for all **console logins** and for **privileged operations** by adding conditions in IAM policies. Example: Block IAM or KMS actions unless MFA is enabled.
- **Choose Strong Factors:** Use **hardware security keys (FIDO2/U2F, e.g., YubiKey)** for maximum assurance. Virtual MFA apps (Google Authenticator, Authy) are acceptable; avoid SMS-based MFA.
- **Granular Enforcement:** Use policy conditions ('aws:MultiFactorAuthPresent') to enforce MFA only on sensitive actions instead of every request.

Best Practice: Treat MFA as a baseline requirement for all developers, admins, and CI/CD pipeline credentials.

Key Rotation and Session Hygiene

- **Rotate Access Keys Automatically** For IAM users and service accounts, schedule automated key rotation. → Prevents long-term key exposure and reduces risk if a key leaks.
- **Prefer Short-Lived Credentials** Use AWS STS (Security Token Service) to issue **temporary credentials** for apps, CI/CD pipelines, and workloads. → Credentials expire (default 1 hour), limiting the blast radius.
- **Enforce Session Tagging** Attach tags (e.g., `team:dev`, `env:prod`) to sessions. → Enables fine-grained access control and improves logging/auditing.
- **Best Practice:** Never hardcode keys in code repos. Store secrets in AWS Secrets Manager or Parameter Store and combine with temporary credentials.

Preventing Privilege Escalation

- **Permission Boundaries** Define the maximum limit of permissions an IAM user/role can ever get. → Even if someone tries to attach an admin policy, the boundary stops it.
- **Service Control Policies (SCPs)** Organization-wide guardrails applied at the AWS Organizations level. → Prevents accounts, users, or roles from ever performing restricted actions (e.g., disabling CloudTrail).
- **Policy Validation Tools** Use **IAM Access Analyzer** and policy simulator to detect risky or unintended access before policies go live. → Reduces the chance of hidden privilege escalation paths.

Best Practice: Combine boundaries (per identity), SCPs (per organization), and analyzers (validation) for layered defense.

Tagging and ABAC

- **Tagging Strategy** Apply consistent tags (key–value pairs) to both identities and resources. Examples: `team:frontend`, `environment:dev`, `owner:alice`.
- **What is ABAC?** Attribute-Based Access Control (ABAC) uses these tags in policies. → Instead of hardcoding resources, policies evaluate tags dynamically.
- **How it works** Use condition keys like `aws:RequestTag` or `aws:TagKeys` in IAM policies. Example: A role tagged `team:frontend` can only access resources tagged `team:frontend`.
- **Why use ABAC?**
 - Reduces the need for dozens of separate roles.
 - Scales easily as teams/projects grow.
 - Enforces least privilege without constant role updates.

Key Idea: One generic policy + consistent tags = scalable, fine-grained access control.

AWS Organizations and SCPs

- **AWS Organizations** Central service to manage **multiple AWS accounts** at once. → Provides consolidated billing, account grouping (OUs), and centralized policy enforcement.
- **Service Control Policies (SCPs)** Organization-level guardrails that define the **maximum permissions** any IAM user or role in member accounts can have. → Even administrators inside an account cannot exceed SCP restrictions.
- **How SCPs work**
 - SCPs do not grant permissions on their own.
 - They only **limit** what IAM policies can allow.
 - Example: An SCP that denies `iam:*` ensures no account can create or modify IAM users/roles.
- **Best Practice** Combine SCPs (org-level guardrails) with IAM policies (account-level permissions) → layered defense that scales across environments.

- **Compliance Alignment** Map your IAM design to industry standards such as **ISO 27001**, **SOC 2**, **HIPAA**. → Ensures security practices meet regulatory and customer requirements.
- **Audit Trail with CloudTrail** Enable AWS CloudTrail in all accounts and regions. → Captures every API call (who, what, when, from where) for investigation, audits, and incident response.
- **Ongoing IAM Health Checks**
 - **IAM Access Analyzer** → flags overly broad or external access.
 - **Credential Reports** → lists user accounts, password/key age, MFA enabled/disabled.→ Run these regularly to detect misconfigurations and stale credentials.
- **Best Practice** Automate compliance checks (e.g., AWS Config, Security Hub) so deviations are detected quickly and remediated before audit deadlines.

Do & Don't Cheat Sheet

✓Do

Use **IAM Roles** for applications and services

Scope policies tightly to **specific resources (ARNs)**

Enforce **MFA for all users and admins**

Rotate keys and use **short-lived STS sessions**

Implement **ABAC with tags** for scalable access control

✗Don't

Create long-term IAM users with static keys for apps

Use wildcards '*' for actions or resources

Leave the root account without MFA enabled

Hardcode permanent access keys in code or config files

Create dozens of nearly identical roles per team/project

Tip: Follow these "Do's" as baseline rules for every AWS environment — they prevent 90% of common IAM misconfigurations.

CLI: Create a Role for EC2

Step 1: Create a trust policy for EC2

```
cat > trust.json <<'EOF'
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"Service": "ec2.amazonaws.com"},
    "Action": "sts:AssumeRole"
  }]
}
EOF
```

Step 2: Create the IAM role using the trust policy

```
aws iam create-role \
--role-name webapp-ec2-role \
--assume-role-policy-document file://trust.json
```

Step 3: Attach a fine-grained inline policy

```
aws iam put-role-policy \
--role-name webapp-ec2-role \
--policy-name S3ReadLogs \
--policy-document file://s3-read-logs.json
```


Explaining: Create a Role for EC2 (CLI)

- **Goal:** Allow an EC2 instance to access AWS resources without hardcoding long-term IAM keys.
- **Step 1 — Trust Policy** (`trust.json`): Defines who can assume the role. Example: `ec2.amazonaws.com` (the EC2 service) is trusted to assume it.
- **Step 2 — Create Role** (`aws iam create-role`): Creates the IAM role (`webapp-ec2-role`) and attaches the trust policy. Note: At this stage, the role has **no permissions**.
- **Step 3 — Attach Permissions** (`aws iam put-role-policy`): Adds an inline policy (e.g., `S3ReadLogs`) granting specific access (e.g., `s3:GetObject` on a log bucket).
- **Step 4 — Launch with Role:** Run EC2 with this role as its instance profile; the instance automatically gets short-lived credentials via the Instance Metadata Service (IMDS).

Launch Command:

```
aws ec2 run-instances \  
--image-id ami-1234567890abcdef \  
--instance-type t2.micro \  
--iam-instance-profile Name=webapp-ec2-role
```

Takeaway: Every IAM Role = Trust Policy (who can assume) + Permissions Policy (what it can do).

Cross-Account Role: Step-by-Step

- ❶ **Step 1 (Account B — Target):** Create an IAM Role and define a **trust policy** that explicitly allows principals from Account A to assume it.
- ❷ **Step 2 (Account B — Target):** Attach a **least-privilege permissions policy** to the role. Example: Allow only `s3:GetObject` on a specific bucket.
- ❸ **Step 3 (Account A — Source):** Create a user or role in Account A and attach a policy that permits calling `sts:AssumeRole` on the ARN of the role in Account B.
- ❹ **Step 4 (Validation):** From Account A, assume the role using CLI/SDK, test access to the target resource, and confirm actions are logged in **CloudTrail**.

Practical Use Case: A CI/CD pipeline in the Dev account assumes a role in the Prod account to deploy applications securely.

Federated Login (Azure AD / Okta)

- **Step 1 — Configure IdP:** Set up your corporate Identity Provider (Azure AD, Okta, etc.) with a SAML or OIDC application for AWS.
- **Step 2 — Map Groups to Roles:** Link enterprise **groups/claims** (e.g., Developers, Admins) to AWS IAM roles using SAML assertions or OIDC tokens.
- **Step 3 — User Login:** Developers log in with their existing corporate credentials (SSO). The IdP authenticates the user and passes a SAML/OIDC response to AWS.
- **Step 4 — AWS STS Issues Temporary Credentials:** AWS Security Token Service (STS) provides short-lived access tokens for the mapped IAM role.

Benefit: Centralized identity management — no need to create or maintain individual IAM users in every AWS account.

EKS: IAM Roles for Service Accounts (IRSA)

- **What it is:** A feature that links **Kubernetes service accounts** to IAM roles using an **OIDC identity provider**.
- **How it works:** When a pod uses a service account, EKS injects temporary credentials from the mapped IAM role into the pod.
- **Why it matters:** Provides **fine-grained AWS permissions per pod**, instead of sharing credentials at the EC2 node level.
- **Security Benefit:** Enforces least privilege at pod level → if one pod is compromised, it only exposes the permissions assigned to that pod.

Use Case: A logging pod needs `s3:PutObject` to write logs to S3 — with IRSA, only that pod gets the permission, not the entire node.

- **Dedicated Roles for Pipelines** Create separate IAM roles for your CI/CD jobs (CodePipeline, GitHub Actions, Jenkins). → Each stage (build, test, deploy) should have its own role.
- **Least Privilege Scoping** Grant only the permissions needed:
 - BuildRole → can compile and push artifacts.
 - DeployRole → can update infrastructure but not production unless approved.
 - ReadRole → read-only for monitoring and reporting.
- **Separation of Duties** Different roles for different actions → reduces blast radius and enforces approval gates.

Best Practice: Never attach AdministratorAccess to CI/CD roles. Use tightly scoped, stage-specific permissions to secure the pipeline.

S3 Access by Role (Example Policy)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListBucketHome",
      "Effect": "Allow",
      "Action": ["s3:ListBucket"],
      "Resource": "arn:aws:s3::my-bucket",
      "Condition": {
        "StringLike": {"s3:prefix": ["home/${aws:username}/*"]}
      }
    },
    {
      "Sid": "HomeDirRW",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3::my-bucket/home/${aws:username}/*"
    }
  ]
}
```

Explanation:

- ListBucketHome → Users can list objects, but only those under their own prefix: `home/${aws:username}/`.
- HomeDirRW → Users can read (GetObject) and write (PutObject) files only inside their personal home folder.
- **Result:** Each IAM principal is isolated to their own directory within the same bucket.

IAM with CloudFormation (Snippet)

```
Resources:
  AppRole:
    Type: AWS::IAM::Role
    Properties:
      AssumeRolePolicyDocument:
        Version: '2012-10-17'
        Statement:
          - Effect: Allow
            Principal:
              Service: [ lambda.amazonaws.com ]
            Action: sts:AssumeRole

    Policies:
      - PolicyName: AppAccess
        PolicyDocument:
          Version: '2012-10-17'
          Statement:
            - Effect: Allow
              Action:
                - dynamodb:GetItem
                - dynamodb:PutItem
              Resource: arn:aws:dynamodb:us-east-1:123456789012:table/Orders
```

Explanation:

- **Trust Policy:** Allows the Lambda service (lambda.amazonaws.com) to assume this role.
- **Permissions Policy:** Grants the role access to perform GetItem and PutItem actions on the DynamoDB table Orders.
- **Result:** Any Lambda function launched with this role can securely read/write items in that table.

Section Recap — Advanced Use Cases

- **Cross-Account Roles** → Enable secure resource sharing between accounts, with least-privilege enforced.
- **Federation (SAML/OIDC)** → Centralize workforce access using enterprise IdPs, avoiding IAM user sprawl.
- **IRSA in EKS** → Grant pods fine-grained AWS permissions, reducing risk compared to node-wide credentials.
- **CI/CD Roles** → Use dedicated roles per pipeline stage (build, deploy, release) to isolate duties and minimize blast radius.
- **Infrastructure as Code (CloudFormation/Terraform)** → Make IAM configuration consistent, repeatable, and auditable.

Key Idea: Advanced IAM = federated identities + temporary credentials + least privilege everywhere.

Case Study: Capital One (2019)

- **What happened?** A misconfigured firewall allowed access to an EC2 instance that had an IAM role with **overly broad permissions**.
- **Impact:** The attacker used those permissions to copy sensitive data from S3 buckets, leading to one of the largest cloud data breaches.
- **Root Cause:** Combination of network misconfiguration + IAM role that violated least-privilege principles.
- **Key Lessons:**
 - Always apply **least privilege** to IAM roles.
 - Use **defense in depth** → IAM guardrails, network controls, monitoring.
 - Continuously monitor with tools like **CloudTrail**, **Access Analyzer**, and anomaly detection.

Takeaway: Misconfigurations at both **network and IAM level** can combine into a critical breach — layered security is essential.

Policy JSON Breakdown

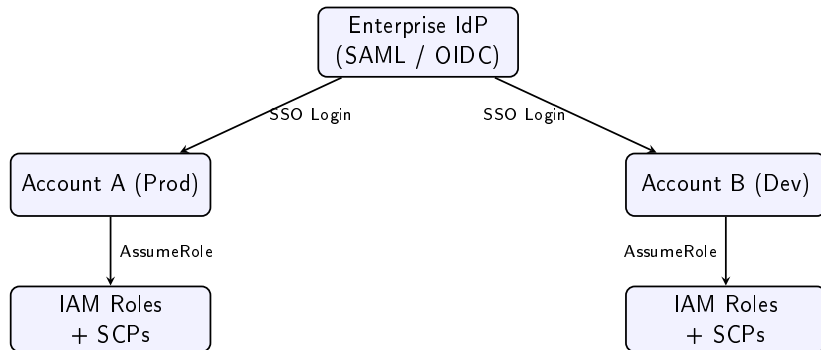
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "RequireMFAForIAM",
      "Effect": "Deny",
      "Action": "iam:*",
      "Resource": "*",
      "Condition": {
        "Bool": { "aws:MultiFactorAuthPresent": "false" }
      }
    }
  ]
}
```

Explanation:

- "Effect": "Deny" → Explicitly denies actions if condition is met.
- "Action": "iam:*" → Applies to all IAM actions (create, update, delete, etc.).
- "Resource": "*" → Applies to all IAM resources in the account.
- "Condition" → Deny applies only when MFA is not present.

Takeaway: This is a **guardrail policy** — users must be logged in with MFA to perform any IAM action.

Reference Architecture (Conceptual)



Explanation:

- **Enterprise IdP** (Azure AD, Okta, etc.) authenticates the user.
- User logs in via **SSO**, mapped to roles in multiple AWS accounts.
- Within each account, **IAM Roles + SCPs** enforce least privilege.
- Users can **AssumeRole** for specific tasks in Prod or Dev securely.

Summary and Next Steps

- **IAM is the foundation** of secure and scalable AWS access.
- Master the core building blocks: **Roles, Policies, Federation, and Guardrails (SCPs, Permission Boundaries)**.
- Always enforce the **principle of least privilege** and use **Infrastructure as Code (IaC)** for consistency.
- Explore advanced tools:
 - **IAM Access Analyzer** → detect unintended access.
 - **Policy Simulator** → test and validate policies safely.
 - **CloudTrail** → monitor activity and build detections.

Next Step: Apply these concepts in a hands-on lab — start by creating roles, testing federation, and validating policies with Access Analyzer.

Further Reading

- **AWS IAM Documentation** — docs.aws.amazon.com/IAM/latest/UserGuide/
- **IAM Access Analyzer Guide** — docs.aws.amazon.com/access-analyzer/latest/userguide/
- **AWS Organizations Guide** — docs.aws.amazon.com/organizations/latest/userguide/
- **AWS Well-Architected Security Pillar** — docs.aws.amazon.com/wellarchitected/latest/security-pillar/wa-security-pillar.html

Tip: Bookmark these pages — they are frequently updated with new IAM features and best practices.